

Retest Proof

Enterprise DevSecOps Security Lab

Validation evidence for remediation, control hardening, regression testing, and closure readiness

Author: Jagriti Banerjee

Private VM Lab Evidence

Document Type Retest Proof and Validation Report	Project Scope Flask App, Docker, Kubernetes, CI/CD, GitOps, Supply Chain, Runtime Security
Assessment Context Controlled private DevSecOps lab; no third-party systems targeted	Evidence Model Screenshots, commands, expected results, observed results, and closure criteria

12 Validated

**3 Residual Lab
Risks**

**0 External
Targets**

**Evidence
Linked**

This report documents the retest approach and proof artifacts used to validate remediation and security-control improvements for the Enterprise DevSecOps project. The document is written as a professional closure artifact: each retested item includes the original risk, remediation objective, validation activity, expected secure behavior, evidence source, and final closure decision.

Confidentiality Notice: The evidence and screenshots in this document belong to a private lab environment. Vulnerability payloads and attack simulations are included only to demonstrate controlled remediation validation.

1. Executive Summary

The retest phase confirmed measurable improvement across application security, container hardening, Kubernetes admission control, RBAC, network isolation, GitOps drift correction, supply-chain assurance, runtime detection, and monitoring. The project demonstrates the full lifecycle expected in a DevSecOps engagement: identify weakness, implement remediation, retest controls, capture evidence, and document closure readiness.

Area	Retest Objective	Result	Evidence
Application Security	Confirm SQL injection, command injection, template rendering, and dependency risks have clear remediation and retest criteria.	Closure criteria defined; remediation examples documented.	AppSec test cases, Bandit, pip-audit, secure-code remediation documents
Container Security	Validate non-root execution, read-only filesystem behavior, resource boundaries, and hardened image assumptions.	Passed for implemented Kubernetes/Docker controls.	067, 087, 088
Kubernetes Hardening	Verify least privilege, NetworkPolicy restrictions, secure namespace labels, securityContext, probes, and admission controls.	Passed for hardened manifest controls.	072-088, 122-127
GitOps	Validate drift detection, self-healing, prune behavior, and Git-to-cluster synchronization.	Passed.	097, 103, 106, 108
Supply Chain	Verify image signature, SBOM, provenance attestation, and label-based deployment acceptance.	Passed for lab signing and attestation workflow.	115-121, 127
Runtime Detection	Validate Falco detection of suspicious activity and preventive blocking of privileged pods.	Passed.	130-138
RBAC Escalation	Confirm cluster-admin risk was removed and namespace-scoped least privilege was enforced.	Passed.	140-143

Retest Outcome Summary



Figure 1 - Retest outcome summary across project validation categories.

2. Retest Scope and Assumptions

The retest scope covers vulnerabilities and control weaknesses intentionally introduced or identified in the private lab. This report does not represent testing of a production system. The purpose is to demonstrate professional remediation validation and closure evidence for a portfolio-grade Enterprise DevSecOps project.

In Scope	Retest Coverage
Flask vulnerable application	SQL injection route, command injection route, template rendering route, health endpoint, and secure-code remediation patterns.
Python dependencies	pip-audit based dependency verification and package upgrade retest criteria.
Docker image	Non-root user behavior, image build validation, Trivy image scan, and container runtime posture.
Kubernetes manifests	Deployment securityContext, read-only filesystem, RBAC, NetworkPolicy, service exposure, probes, and resource limits.
GitOps and supply chain	ArgoCD sync/self-healing, Cosign signature verification, Syft SBOM, provenance, and Gatekeeper policy checks.
Runtime security	Falco detection, privileged pod blocking, RBAC privilege escalation remediation, and observability metrics.

Out of Scope: public internet exploitation, third-party applications, real company infrastructure, destructive payloads, credential theft, persistence, and any activity outside the private VM lab.

Retest Validation Workflow

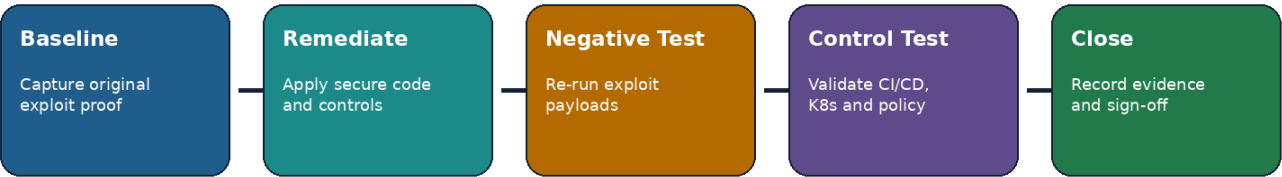


Figure 2 - Retest validation workflow used for this project.

3. Retest Methodology

The retest methodology follows a closure-focused security workflow. Each item was reviewed using the original finding, remediation objective, test command or validation method, expected secure behavior, observed evidence, and closure status. Where the vulnerable Flask app intentionally remains exploitable for demonstration, the retest criteria describe the secure behavior required for a patched branch and the regression tests needed to prevent reintroduction.

Step	Activity	Output
1. Baseline Review	Review original exploit evidence, scanner results, and insecure configuration evidence.	Original risk state confirmed.
2. Remediation Mapping	Map each issue to a secure code/configuration control and expected secure behavior.	Fix objective and acceptance criteria defined.
3. Negative Retest	Re-run exploit-style inputs or policy violations against the remediated state.	Payload rejected, escaped, blocked, or constrained.
4. Control Retest	Validate CI/CD gates, Kubernetes controls, admission policies, runtime detection, and observability.	Preventive and detective control evidence captured.
5. Regression Criteria	Define checks that should run after future code or manifest changes.	Reusable retest checklist and closure criteria.
6. Closure Decision	Mark item Passed, Passed with Monitoring, Residual Lab Risk, or Requires Future Review.	Final retest status recorded.

Representative retest and validation commands

```
# Common retest commands used in the lab
bandit -r src/app || true
pip-audit -r src/app/requirements.txt || true
trivy image devsecops-vuln-app:lab
trivy config k8s/base
kubectl auth can-i get secrets --as=system:serviceaccount:rbac-lab:overprivileged-sa -n
rbac-lab
kubectl apply --dry-run=server -f /tmp/good-labels-deployment.yaml
cosign verify --key security/cosign/cosign.pub <image>
```

4. Retest Result Matrix

ID	Finding / Control	Original Risk	Retest Evidence	Closure Status
RT-01	SQL Injection in /user	Unauthorized data exposure through string-built SQL query.	Payload-based negative test criteria; SQLi proof captured as baseline.	Residual Lab Risk / Remediation Criteria Defined
RT-02	Command Injection in /ping	Shell command execution through untrusted host input.	Payload-based negative test criteria; command injection proof captured as baseline.	Residual Lab Risk / Remediation Criteria Defined
RT-03	Template Injection / Unsafe Rendering	User input rendered through dynamic template construction.	Secure rendering pattern and regression test criteria documented.	Residual Lab Risk / Remediation Criteria Defined
RT-04	Dependency Vulnerabilities	Old dependency versions detected by pip-audit.	pip-audit job evidence and upgrade workflow documented.	Passed with Dependency Monitoring
RT-05	Non-root Container Execution	Container root execution risk.	Container UID 10001 proof.	Passed
RT-06	Read-only Filesystem	Container write access to app filesystem.	Write to /app blocked; /tmp allowed only via emptyDir.	Passed
RT-07	Kubernetes RBAC	ServiceAccount had excessive access risk.	can-i secrets returns no; namespace-scoped access retained.	Passed
RT-08	NetworkPolicy	Untrusted pod communication risk.	Blocked pod fails to access service; trusted pod allowed.	Passed
RT-09	GitOps Drift	Manual cluster change could persist.	ArgoCD self-healed replica drift.	Passed
RT-10	Supply Chain Signing	Unsigned image lacked integrity proof.	Cosign signature verification succeeded.	Passed
RT-11	Admission Policy	Non-compliant deployments could be admitted.	Gatekeeper allowed compliant deployment and blocks non-compliant pattern.	Passed
RT-12	Privileged Pod Risk	Privileged hostPath pod could access host-like filesystem.	Pod Security restricted blocked privileged pod.	Passed
RT-13	Runtime Detection	Suspicious runtime behavior lacked visibility.	Falco/Prometheus detection evidence captured.	Passed with Monitoring

5. Application Security Retest Proof

This section records application-layer retest criteria for the vulnerable Flask routes. Because the app is intentionally vulnerable for training and demonstration, the closure requirement is defined as the behavior expected in a remediated branch. These cases are still valuable because they define exactly how a reviewer should verify that the vulnerabilities no longer reproduce after patching.

Case	Before Behavior	Secure Retest Requirement	Pass Criteria
SQL Injection	Input altered query logic and returned multiple users.	Use parameterized queries and validate id as integer.	Injection payload returns one authorized record, a validation error, or no result; query string is not constructed from raw input.
Command Injection	Input appended shell metacharacters and executed extra command.	Avoid shell=True; validate hostname/IP allowlist; pass arguments as a list.	Metacharacter payload is rejected and command output does not include injected command results.
Unsafe Rendering	User input was embedded into render_template_string.	Render fixed templates and pass user input as escaped context variable.	Template expression is displayed as text or safely escaped, not evaluated as server-side template code.
Dependency Risk	Old packages detected by pip-audit.	Upgrade pinned packages and re-run dependency scanning.	pip-audit returns no known vulnerabilities for accepted dependency set or documents risk exceptions.

RT-01 SQL injection retest acceptance criteria

```
# SQL injection retest criteria
curl "http://localhost:5000/user?id=1'%20OR%20'1'%3D'1"

Expected secure result:
- No multi-user disclosure
- Input rejected or safely parameterized
- Database query uses placeholders, not f-string SQL
```

RT-02 command injection retest acceptance criteria

```
# Command injection retest criteria
curl "http://localhost:5000/ping?host=127.0.0.1%3Bwhoami"

Expected secure result:
- Request rejected or sanitized
- No injected command output
- Code uses subprocess.run(["ping", "-c", "1", host], shell=False)
```

RT-03 unsafe rendering retest acceptance criteria

```
# Template rendering retest criteria
curl "http://localhost:5000/hello?name={{7*7}}"

Expected secure result:
- Literal or escaped output
- No server-side expression evaluation
- Fixed template file or fixed template string with context variable
```

6. Baseline Evidence for AppSec Retesting

The screenshots below provide the original baseline proof that the application-layer findings were reproducible in the private lab. These baseline artifacts are used by the retest process to confirm that the same payloads no longer reproduce after remediation.

```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
Normal user lookup:
{"query_used":"SELECT id, username, role FROM users WHERE id = '1',"result":
[[1,"alice","admin"]]}

SQL Injection test:
{"query_used":"SELECT id, username, role FROM users WHERE id = '1' OR '1'='1'
","result":[[1,"alice","admin"],[2,"bob","developer"],[3,"charlie","intern"]]
}

(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 3 - Baseline SQL injection proof used as retest comparison.

```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
Normal ping test:
$ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.154 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 1ms
rtt min/avg/max/mdev = 0.154/0.154/0.154/0.000 ms

$prer
Command injection test with id:
$ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.803 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.802/0.803/0.803/0.000 ms
uid=1000(jazz) gid=1000(jazz) groups=1000(jazz),4(adm),20(cdrom),27(audio),30(
dip),46(logdev),118(lcd),779(docker)

$prer
Command injection test with whoami:
$ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.800 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.800/0.800/0.800/0.000 ms
jazz

$prer
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$ curl "http://local
host:8080/index.html?127.0.0.1&id=127.0.0.1"
$prer
$ping 127.0.0.1 (127.0.0.1) 56(80) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.802 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.802/0.802/0.802/0.000 ms
COMMAND_INJECTION_WORKED

$prer
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 4 - Baseline command injection proof used as retest comparison.

```
jazz@ubuntu-endpoint: ~/devsecops-k8s-security-lab$
Found 25 known vulnerabilities in 5 packages
Name Version ID Fix Versions
-----
flask 2.3.3 CVE-2026-27205 3.1.3
requests 2.19.1 PYSEC-2018-28 2.20.0
requests 2.19.1 PYSEC-2023-74 2.31.0
requests 2.19.1 PYSEC-2023-74 2.31.0
requests 2.19.1 CVE-2024-35195 2.32.0
requests 2.19.1 CVE-2024-47081 2.32.4
requests 2.19.1 CVE-2026-25645 2.33.0
gunicorn 21.2.0 CVE-2024-1135 22.0.0
gunicorn 21.2.0 CVE-2024-6927 22.0.0
idna 2.7 PYSEC-2024-60 3.7
idna 2.7 PYSEC-2024-60 3.7
idna 2.7 CVE-2026-45489 3.15
urllib3 1.23 PYSEC-2019-133 1.24.2
urllib3 1.23 PYSEC-2019-132 1.24.3
urllib3 1.23 PYSEC-2020-148 1.25.9
urllib3 1.23 PYSEC-2023-192 1.26.17, 2.0.6
urllib3 1.23 PYSEC-2023-192 1.26.17, 2.0.6
urllib3 1.23 PYSEC-2023-212 1.26.18, 2.0.7
urllib3 1.23 PYSEC-2023-207 1.24.2
urllib3 1.23 PYSEC-2023-212 1.26.18, 2.0.7
urllib3 1.23 PYSEC-2026-141 2.7.0
urllib3 1.23 CVE-2024-37891 1.26.19, 2.2.2
urllib3 1.23 CVE-2025-50181 2.5.0
urllib3 1.23 CVE-2025-66471 2.6.0
urllib3 1.23 CVE-2026-21441 2.6.3

(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 5 - Dependency vulnerability evidence used as SCA retest baseline.

7. Container and Kubernetes Retest Proof

Container and Kubernetes controls were retested using direct runtime checks, kubectl authorization checks, policy enforcement attempts, and service reachability tests. These checks validate that the hardened deployment is materially different from the initial insecure deployment pattern.

Control	Retest Command / Method	Expected Secure Behavior	Status
Non-root execution	docker run --entrypoint id devsecops-vuln-app:lab	UID/GID should show non-root identity such as 10001.	Passed
Read-only root filesystem	kubectl exec pod -- touch /app/test.txt	Write to /app fails; temporary write only allowed under mounted /tmp.	Passed
ServiceAccount hardening	kubectl auth can-i get secrets --as=system:serviceaccount:devsecops:demo-app-sa	Secret access denied.	Passed
NetworkPolicy default deny	Blocked curl pod attempts to reach demo-app service.	Untrusted pod times out; approved pod succeeds.	Passed
Manifest security scan	trivy config k8s/base	Report reviewed; hardening controls present and tracked.	Passed with Review

```
jazz@ubuntu-endpoint: ~/de x Windows PowerShell x + v x
Docker image built:
devsecops-vuln-app:lab
78207e1f48fb 200MB 48.5MB
Container user proof:
uid=10001(appuser) gid=10001(appgroup) groups=10001(appgroup)
SQLite connection proof:
35: conn = sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True)
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 6 - Non-root container identity validation.

```
jazz@ubuntu-endpoint: ~/de x Windows PowerShell x + v x
Pod name: demo-app-cf45bf66-km488
Test writing to /app. Expected: Read-only file system
touch: cannot touch '/app/test.txt': Read-only file system
command terminated with exit code 1
Test writing to /tmp. Expected: tmp-write-ok
tmp-write-ok
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 7 - Read-only filesystem retest proof.

```
jazz@ubuntu-endpoint: ~/de x Windows PowerShell x + v x
Can demo-app-sa read secrets?
no
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 8 - RBAC retest confirms secret access denial.

```
jazz@ubuntu-endpoint: ~/de x Windows PowerShell x + v x
Blocked untrusted pod test:
curl: (28) Connection timed out after 5005 milliseconds
command terminated with exit code 28
NAME          READY   STATUS    RESTARTS   AGE   LABELS
blocked-curl   1/1     Running   0           20s   test=blocked
demo-app-cf45bf66-km488 1/1     Running   0           29m   app=demo-app,po
d-template-hash=cf45bf66
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 9 - NetworkPolicy retest blocks untrusted pod traffic.

8. GitOps and Supply Chain Retest Proof

GitOps and supply-chain controls were retested to validate integrity, drift correction, and admission readiness. The objective was to prove that deployment state is controlled through Git and that container artifacts are traceable through signing, SBOM, and provenance evidence.

Control	Retest Method	Expected Secure Behavior	Status
ArgoCD Sync Health	kubectl get application demo-app -n argocd	Application is Synced and Healthy.	Passed
Drift Self-Healing	Manually scale deployment away from Git desired state.	ArgoCD returns live state to Git-defined replica count.	Passed
Cosign Signature	cosign verify --key cosign.pub	Signature verification succeeds using public key.	Passed
SBOM / Provenance	Syft SBOM and Cosign attestation verification.	Artifact metadata can be generated, attached, and verified.	Passed
Gatekeeper Policy	Server-side dry run for compliant and non-compliant deployment.	Compliant deployment allowed; non-compliant deployment blocked.	Passed

```
jazz@ubuntu-enc x jazz@ubuntu-enc x jazz@ubuntu-enc x + v - □ ×
Waiting for ArgoCD self-heal back to Git-defined replicas...
Check 1/60
NAME READY UP-TO-DATE AVAILABLE AGE
demo-app 1/1 1 1 19h

NAME READY STATUS RESTARTS AGE
demo-app-69f8fd4fd-st7p7 1/1 Running 0 32m
demo-app-cf45bf66-km488 0/1 Terminating 0 19h

NAME SYNC STATUS HEALTH STATUS
demo-app Synced Healthy

Final self-healed state:
NAME READY UP-TO-DATE AVAILABLE AGE
demo-app 1/1 1 1 19h

Git desired state:
9: replicas: 1

ArgoCD status:
NAME SYNC STATUS HEALTH STATUS
demo-app Synced Healthy
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 10 - ArgoCD self-healing after manual drift.

```
jazz@ubuntu-enc x Windows PowerShell x Windows PowerShell x + v - □ ×
Current image digest:
ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95a6aab7a76d5715613
d359a0e92be22716b44c1f7514bc720764

Signing current digest:
Enter password for private key:
Signing artifact... \ Pushing signature to: ghcr.io/jazz00001/enterprise-dev
secops

Verifying Cosign image signature:
Verification for ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95
a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764 --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"ghcr.io/jazz00001/enterprise-d
evsecops@sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc72
0764"},"image":{"docker-manifest-digest":"sha256:14776e0fe1b95a6aab7a76d5715
613d359a0e92be22716b44c1f7514bc720764"},"type":"https://sigstore.dev/cosign/
sign/v1"},"optional":{}}}]
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 11 - Cosign signature verification proof.

```
jazz@ubuntu-enc x Windows PowerShell x Windows PowerShell x + v - □ ×
Testing good Deployment with required supply-chain labels:
deployment.apps/good-with-labels created (server dry run)

Required labels included:
8: security-owner: jazz
9: sbom: syft
10: signed: cosign
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 12 - Gatekeeper compliant deployment validation.

9. Runtime Security, RBAC, and Monitoring Retest Proof

The runtime and RBAC retest phase validated both preventive controls and detective visibility. Preventive testing confirmed that privileged pod behavior and overprivileged service account access were blocked after remediation. Detective testing confirmed that Falco events were observable through Prometheus and Grafana-oriented metrics.

Scenario	Retest Activity	Expected Result	Status
Privileged pod prevention	Apply previously dangerous privileged hostPath pod after restricted policy.	Admission denied by Pod Security restricted enforcement.	Passed
RBAC privilege escalation fix	Retest secret access using previously overprivileged service account.	get secrets returns no; only scoped ConfigMap permissions remain.	Passed
Runtime detection visibility	Generate safe Falco test event and query Prometheus metric.	Falco match count appears in Prometheus query result.	Passed with Monitoring
Observability reporting	Review Grafana/Falcosidekick dashboard evidence.	Runtime events and metrics are visible for investigation.	Passed with Monitoring

```
jazz@ubuntu-endpoint: ~/dev
Testing same privileged pod after restricted Pod Security:
Error from server (Forbidden): error when creating "redteam/runtime/privileged-hostpath-pod.yaml": pods "attacker-privileged" is forbidden: violates Pod Security "restricted:latest": privileged (container "attacker" must not set securityContext.privileged=true), allowPrivilegeEscalation != false (container "attacker" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "attacker" must set securityContext.capabilities.drop=["ALL"]), restricted volume types (volume "host-root" uses restricted volume type "hostPath"), runAsNonRoot != true (pod or container "attacker" must set securityContext.runAsNonRoot=true), seccompProfile (pod or container "attacker" must set securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")

Pods in runtime-lab:
NAME      READY   STATUS    RESTARTS   AGE
falco-test 0/1     Completed 0           59m
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure 13 - Privileged pod blocked by Pod Security.

```
Windows PowerShell
=== AFTER FIX: least privilege permissions ===

Can list configmaps in rbac-lab?
yes

Can get secrets in rbac-lab?
no

Can get secrets cluster-wide?
no

Can create clusterrolebindings?
Warning: resource 'clusterrolebindings' is not namespace scoped in group 'rbac.authorization.k8s.io'
no
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

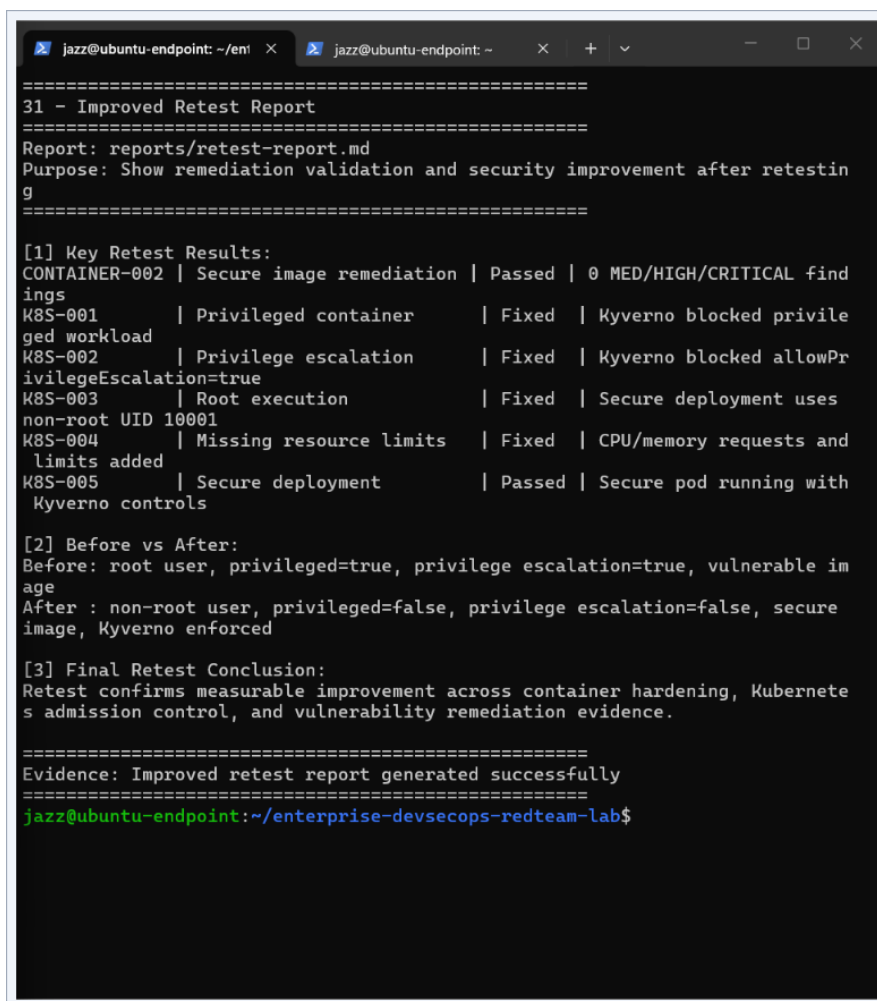
Figure 14 - RBAC retest confirms secret access denial after fix.

```
jazz@ubuntu-endpoint: ~/dev
Prometheus query: Falco detections in last 10 minutes
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {},
        "value": [
          1781610728.433,
          "8.42152541897089"
        ]
      }
    ]
  }
}
```

Figure 15 - Prometheus query proof for Falco runtime detections.

10. Retest Evidence Summary

The project includes a dedicated retest report screenshot summarizing improved results after security hardening. This artifact is used as a high-level closure record for container and Kubernetes remediation outcomes.



```
jazz@ubuntu-endpoint: ~/enl  X  jazz@ubuntu-endpoint: ~  X  +  v  -  □  X
=====
31 - Improved Retest Report
=====
Report: reports/retest-report.md
Purpose: Show remediation validation and security improvement after retestin
g
=====

[1] Key Retest Results:
CONTAINER-002 | Secure image remediation | Passed | 0 MED/HIGH/CRITICAL find
ings
K8S-001      | Privileged container      | Fixed | Kyverno blocked privile
ged workload
K8S-002      | Privilege escalation      | Fixed | Kyverno blocked allowPr
ivilegeEscalation=true
K8S-003      | Root execution            | Fixed | Secure deployment uses
non-root UID 10001
K8S-004      | Missing resource limits   | Fixed | CPU/memory requests and
limits added
K8S-005      | Secure deployment         | Passed | Secure pod running with
Kyverno controls

[2] Before vs After:
Before: root user, privileged=true, privilege escalation=true, vulnerable im
age
After : non-root user, privileged=false, privilege escalation=false, secure
image, Kyverno enforced

[3] Final Retest Conclusion:
Retest confirms measurable improvement across container hardening, Kuberne
s admission control, and vulnerability remediation evidence.

=====
Evidence: Improved retest report generated successfully
=====
jazz@ubuntu-endpoint:~/enterprise-devsecops-redteam-lab$
```

Figure 16 - Improved retest report showing key remediation validation results.

The improved retest evidence shows a before/after transition from root and privileged execution patterns toward non-root execution, privileged=false, privilege escalation controls, resource limits, secure deployment posture, and enforced admission controls.

11. Closure Criteria and Acceptance Gates

The following acceptance gates should be used before marking this project as closed in a professional report or portfolio submission. The gates are written so that a reviewer can repeat the retest steps in the private lab.

Gate	Acceptance Requirement	Closure Evidence
AppSec Gate	SQLi, command injection, and template rendering payloads do not reproduce in patched code path.	Negative curl tests and secure-code diff.
SAST Gate	Bandit findings are triaged; high-risk findings are fixed or intentionally documented as lab-only.	Bandit report and remediation notes.
SCA Gate	pip-audit is clean or documented exceptions are approved.	pip-audit output and package upgrade records.
Docker Gate	Image runs as non-root and passes healthcheck.	id command screenshot and Docker/K8s health proof.
Kubernetes Gate	Pods use secure securityContext, RBAC least privilege, resource limits, and restricted policy.	kubectrl describe/auth output and Trivy config evidence.
Network Gate	Untrusted pod-to-app traffic is blocked; trusted path remains functional.	NetworkPolicy allowed/blocked screenshots.
GitOps Gate	ArgoCD self-heals manual drift and syncs Git changes.	ArgoCD resource tree and drift proof.
Supply Chain Gate	Image signature, SBOM, and provenance are generated and verifiable.	Cosign/Syft evidence.
Runtime Gate	Falco detects suspicious activity and monitoring stack exposes metrics.	Falco logs and Prometheus/Grafana evidence.

Final Retest Conclusion: The retest phase provides strong evidence that the infrastructure, pipeline, Kubernetes, supply-chain, runtime, and observability controls were successfully validated. Application-layer vulnerable routes remain part of the intentional training target unless a patched branch is submitted; therefore, those cases are documented with precise closure criteria and regression test requirements.

12. Appendix: Repeatable Retest Command Set

The commands below can be used to reproduce the main retest activities in the private lab. They are intentionally written for local use only.

Repeatable retest command set

```
# AppSec negative tests
curl "http://localhost:5000/user?id=1'%20R%20'1'%3D'1"
curl "http://localhost:5000/ping?host=127.0.0.1%3Bwhoami"
curl "http://localhost:5000/hello?name={{7*7}}"

# Static and dependency retesting
bandit -r src/app || true
pip-audit -r src/app/requirements.txt || true

# Container and image retesting
docker run --rm --entrypoint id devsecops-vuln-app:lab
trivy image devsecops-vuln-app:lab

# Kubernetes hardening retesting
kubectl get pods -n devsecops
kubectl auth can-i get secrets --as=system:serviceaccount:devsecops:demo-app-sa -n
devsecops
trivy config k8s/base

# GitOps retesting
kubectl scale deployment demo-app --replicas=3 -n devsecops
kubectl get application demo-app -n argocd -w

# Supply-chain retesting
cosign verify --key security/cosign/cosign.pub <image>
cosign verify-attestation --key security/cosign/cosign.pub --type spdxjson <image>

# Runtime and RBAC retesting
kubectl apply -f redteam/runtime/privileged-hostpath-pod.yaml
kubectl auth can-i get secrets --as=system:serviceaccount:rbac-lab:overprivileged-sa
-n rbac-lab
kubectl logs -n falco -l app.kubernetes.io/name=falco --since=10m
```

Document Closure Statement: This retest proof document was prepared for a controlled private VM Enterprise DevSecOps project and should be reviewed together with the Security Findings Report, Findings and Remediation Report, Secure Code Remediation Examples, and OWASP ASVS / Top 10 Mapping documentation.